

Git Internals

From first principles to production-grade large-repo optimization

TOPICS COVERED

- › History & Design Philosophy
- › Object Model: Blob / Tree / Commit / Tag
- › Object Database & Pack Files
- › Staging Area, Index & Working Tree
- › Cherry-Pick, Reset, Restore, Revert
- › Git Garbage Collection
- › Sparse Checkout & Partial Clone
- › Subtree Merges
- › Large Repository Optimization
- › Distributed VCS Architecture
- › SHA-1 → SHA-256 Migration
- › Reflog & HEAD mechanics
- › Rebase & Merge Internals
- › Interactive Rebase
- › Git Hooks
- › Git LFS & Submodules
- › Monorepo Support

1.1 History of Git

Git was created by Linus Torvalds in April 2005 after the Linux kernel community lost access to BitKeeper, the proprietary DVCS they had been using. The kernel needed a system that could handle tens of thousands of contributors, branch at the speed of thought, and guarantee data integrity through cryptographic hashing. Torvalds wrote the first version in roughly two weeks, with the explicit goal of being the "stupid content tracker" — minimal, fast, and correct. Within months Junio Hamano took over as maintainer, a role he holds to this day.

The name 'Git' is British slang for an unpleasant person. Torvalds described it as 'the stupid content tracker' — he named all his projects after himself: Linux and Git.

■ *Key design goals: speed, data integrity, support for distributed non-linear workflows, and explicit design for large projects.*

Why Distributed?

Previous generation VCS tools (CVS, Subversion) required a central server for almost every operation. Git makes every clone a full-fledged repository with complete history, enabling:

- Offline commits and history browsing
- Parallel development without central server availability
- Faster operations (most actions are local disk reads)
- Multiple remotes and complex fork/merge topologies
- Cryptographic verification of every object in the history

1.2 The Git Object Model

Git is, at its core, a content-addressable key-value store built on four immutable object types. Every object is identified by the SHA-1 (or SHA-256) hash of its content. This means identical content always produces the same hash — Git never stores the same data twice.

Blob

A blob (Binary Large Object) stores raw file content. It knows nothing about filenames or directory structure — it is purely content.

```
# How Git stores a blob internally:
$ echo 'Hello, World!' | git hash-object --stdin
8ab686eafeb1f44702738c8b0f24f2567c36da6d

# The blob format:
# "blob <size>\0<content>"
# SHA1( "blob 14\0Hello, World!\n" ) = 8ab686...

# To inspect an object:
$ git cat-file -t 8ab686      # → blob
$ git cat-file -p 8ab686      # → Hello, World!
```

■ *Blobs are deduplicated automatically. If 1,000 files share the same content, Git stores only one blob.*

Tree

A tree object represents a directory. It contains references to blobs (files) and other trees (subdirectories), along with file modes and names.

```
# Inspect a tree object:
$ git cat-file -p HEAD^{tree}
100644 blob a8c9f... README.md
100755 blob 3b2c1... deploy.sh
040000 tree f1d20... src/
040000 tree 9a3b7... tests/

# File mode meanings:
# 100644 = regular file
# 100755 = executable file
# 120000 = symbolic link
# 040000 = directory (tree)
# 160000 = gitlink (submodule)
```

Commit

A commit object ties a tree (the root directory snapshot) to author/committer metadata and zero or more parent commits. This forms the DAG (Directed Acyclic Graph) that represents history.

```
$ git cat-file -p HEAD
tree f1d20c8a3b...
parent 9c3d7f2e1a...
author Jane Smith <jane@co.com> 1686000000 +0000
committer Jane Smith <jane@co.com> 1686000000 +0000

feat: add OIDC authentication

- Replaces long-lived tokens with short-lived OIDC tokens
- Integrates with AWS IAM, Azure Entra ID, GCP Workload Identity
- Zero stored secrets in GitHub Actions

Refs: #1234
Signed-off-by: Jane Smith <jane@co.com>
```

Each commit is a complete snapshot of the entire repository, not a diff. Git reconstructs diffs by comparing adjacent trees on the fly. This is why Git can efficiently checkout any commit in $O(1)$ tree-pointer lookups rather than replaying patches.

Tag Object

Lightweight tags are simply refs pointing to commits. Annotated tags are first-class objects with their own metadata, signature, and message.

```
$ git cat-file -p v2.0.0
object 9c3d7f2e1a...
type commit
tag v2.0.0
tagger Jane Smith <jane@co.com> 1686001234 +0000

Release 2.0.0 - Major OIDC refactor
-----BEGIN PGP SIGNATURE-----
...
-----END PGP SIGNATURE-----
```

■ *Always use annotated tags for releases — they carry a GPG signature, tagger identity, and timestamp. Lightweight tags are just named pointers and carry none of this.*

1.3 SHA-1 vs SHA-256 Migration

Git has historically used SHA-1 as its object identifier. While SHA-1 is cryptographically broken for signature use cases (SHAttered attack, 2017), Git's use is as a content integrity check — a deliberate collision requires

computational resources that make it impractical for most attack scenarios. Nevertheless, the Git community began migrating to SHA-256 (SHA-2 256-bit).

The 'New Hash' format (sha256) was introduced in Git 2.29 (2020). GitHub and major forges are in the process of supporting SHA-256 repositories.

```
# Initialize a new SHA-256 repository:
$ git init --object-format=sha256 my-repo
# SHA-256 object IDs are 64 hex characters:
# e.g. a7d8f3... (64 chars) vs SHA-1's 40 chars

# Check object format:
$ git rev-parse --show-object-format
sha256
```

■ *SHA-256 repositories are not backwards-compatible with SHA-1. Interoperability requires translation layers. At time of writing, most enterprise tooling still defaults to SHA-1.*

1.4 Object Database & Pack Files

Loose Objects

Newly created objects are stored as individual compressed files under `.git/objects/`. The first two hex characters of the SHA form a subdirectory:

```
.git/objects/
 8a/b686eafefb1f44702738c8b0f24f2567c36da6d ← blob
9c/3d7f2e1a...                               ← commit
f1/d20c8a3b...                               ← tree
```

Pack Files

As repositories grow, thousands of loose objects become inefficient. Git consolidates them into pack files — binary archives containing delta-compressed objects. This is where Git's storage efficiency comes from.

```
.git/objects/pack/
 pack-abc123.idx   ← index for fast offset lookup
 pack-abc123.pack  ← compressed delta objects

# Manually trigger packing (also done automatically):
$ git gc
# Or more aggressively:
$ git gc --aggressive --prune=now

# Inspect pack contents:
$ git verify-pack -v .git/objects/pack/pack-abc123.idx | head -20
```

Delta compression in pack files stores the difference between similar objects rather than full copies. Git selects delta bases by content similarity, not file lineage — a copy of a file in another directory can serve as a delta base for the original.

Pack Index Format (v2)

The `.idx` file provides $O(1)$ lookup of any object. It contains a fan-out table (256 entries), sorted object IDs, CRCs, and offsets into the `.pack` file. Large packs use 64-bit offsets for files >2GB.

1.5 Reflog

The reflog is a local safety net that records every change to a ref (branch pointer, HEAD). It is NOT part of the repository history and is NOT pushed to remotes.

```
$ git reflog
abc1234 HEAD@{0}: commit: feat: add OI DC
9c3d7f2 HEAD@{1}: checkout: moving from main to feature/oidc
fd20c8 HEAD@{2}: merge feature/login: Merge made by 'ort'
...

# Recover a dropped commit:
$ git checkout -b recovery HEAD@{3}

# Reflog has configurable expiry:
# gc.reflogExpire = 90 days (default)
# gc.reflogExpireUnreachable = 30 days
```

■ *git gc --prune=now permanently deletes unreachable objects. Always check reflog before aggressive GC on production repositories.*

1.6 HEAD and Detached HEAD

HEAD is a symbolic ref that usually points to a branch name, which in turn points to a commit:

```
# Normal state:
$ cat .git/HEAD
ref: refs/heads/main

# Detached HEAD (points directly to a commit SHA):
$ git checkout abc1234
$ cat .git/HEAD
abc1234... ← SHA-1 directly

# This is dangerous: new commits made in detached HEAD
# are not reachable from any branch and will be garbage-collected.
# Always create a branch to preserve work:
$ git checkout -b save-my-work
```

1.7 Staging Area (Index)

The index is a binary file (.git/index) that acts as the "proposed next commit." It is the key to understanding how Git separates content selection from commit creation.

```
# Three-way relationship:
# Working Tree ↔ Index (Staging Area) ↔ Repository

$ git add file.py      # Working Tree → Index
$ git commit           # Index → Repository (new commit object)
$ git checkout file.py # Repository → Working Tree (via Index)

# Inspect index in detail:
$ git ls-files --stage
100644 a8c9f... 0 README.md
100644 3b2c1... 0 src/main.py
#           ↑ stage number (0=normal, 1-3=merge conflict stages)
```

During a merge conflict, the index holds three versions simultaneously: stage 1 (common ancestor), stage 2 (ours), stage 3 (theirs). Tools like git mergetool use these to present a 3-way diff.

1.8 Rebase Internals

Rebase replays commits from a source branch onto a new base, creating brand-new commit objects with new SHAs. The original commits remain until garbage collected.

```
# Before rebase:
#   A -- B -- C  (main)
#               \
#               D -- E  (feature)

$ git checkout feature
$ git rebase main

# Git internally:
# 1. Find common ancestor: B
# 2. Extract patches: D-patch, E-patch  (git format-patch)
# 3. Apply each patch onto main's tip C:
#   C -- D' -- E'  (feature, new SHAs!)
# 4. Move feature branch pointer to E'

# After rebase:
#   A -- B -- C -- D' -- E'  (feature, rebased)
#               ^
#               (main still here)
```

■ *NEVER rebase commits already pushed to a shared branch. It rewrites history (new SHAs) causing divergence for all other developers. The golden rule: never rebase public history.*

Rebase vs Merge — Decision Matrix

Criterion	Rebase	Merge
History shape	Linear, clean	Preserves branch topology
Commit SHAs	Changed (new objects)	Preserved
Shared branches	NEVER safe	Always safe
PR squash	Common pattern	Merge commit alternative
Bisect friendliness	Excellent (linear)	Good (bisect handles merges)
Blame accuracy	Requires --follow	Accurate
Enterprise policy	Feature branches only	Main/release branches

1.9 Merge Internals

Git's default merge strategy is 'ort' (Ostensibly Recursive's Twin) since Git 2.34, replacing the previous 'recursive' strategy. It resolves three-way merges by finding the best common ancestor using the merge base algorithm.

```
# Merge strategies:
$ git merge --strategy=ort feature           # default
$ git merge --strategy=octopus feat1 feat2  # >2 branches
$ git merge --strategy=ours feature          # ignore theirs entirely
$ git merge --strategy=subtree feature       # prefix tree

# Merge options:
$ git merge --no-ff feature                  # always create merge commit
$ git merge --ff-only feature                # fail if not fast-forward
$ git merge --squash feature                 # stage all changes, no commit
```

Merge Conflict Anatomy

```
<<<<<<< HEAD (ours)
      return authenticate_oidc(user)
=====
```

```

    return authenticate_saml(user)
>>>>>> feature/saml-auth (theirs)
# Common ancestor had: return authenticate_basic(user)

```

1.10 Cherry-Pick, Reset, Restore, Revert

Cherry-Pick

Applies a single commit's changes as a new commit onto the current branch. Internally: extracts the diff between the commit and its parent, then applies it.

```

$ git cherry-pick abc1234          # single commit
$ git cherry-pick abc1234..def5678 # range (exclusive start)
$ git cherry-pick -x abc1234       # append original SHA to message
$ git cherry-pick --no-commit abc1234 # stage only, don't commit

```

Reset vs Restore vs Revert

Command	Affects	Safe for Shared?	Use Case
git reset --soft SHA	HEAD only	No	Undo commit, keep staged
git reset --mixed SHA	HEAD + Index	No	Undo commit, unstage
git reset --hard SHA	HEAD+Index+WorkTree	No	Nuke everything to SHA
git restore file	Working tree	Yes	Discard local changes
git restore --staged file	Index	Yes	Unstage a file
git revert SHA	Creates new commit	Yes	Undo on shared branches

■ *git reset --hard is destructive. Use git revert for commits already on shared/protected branches.*

1.11 Interactive Rebase

Interactive rebase (git rebase -i) allows rewriting history by reordering, squashing, editing, or dropping commits before they're shared. This is the standard tool for 'cleaning up' a feature branch before a PR.

```

$ git rebase -i HEAD~5

# Editor opens:
pick a1b2c3 feat: add login form
pick d4e5f6 fix typo
pick 7890ab wip: debug logging          ← will squash
pick cd1234 fix: remove debug logging ← will drop after squash
pick ef5678 feat: add OIDC callback

# Commands:
# p/pick    = use commit as-is
# r/reword  = use commit, edit message
# e/edit    = use commit, stop to amend
# s/squash  = meld into previous commit
# f/fixup   = like squash but discard this commit's message
# d/drop    = remove commit entirely
# x/exec    = run shell command after this line

```

1.12 Git Garbage Collection

```
# Automatic GC triggers when loose objects exceed gc.auto (default 6700)
# or loose packs exceed gc.autoPackLimit (default 50)

$ git gc          # standard cleanup
$ git gc --prune=now # remove unreachable objects immediately (dangerous)
$ git gc --aggressive # more CPU, better delta compression

# Maintenance (preferred for large repos):
$ git maintenance start # enables background scheduled tasks
# Runs: prefetch, loose-objects, incremental-repack, commit-graph, pack-refs

# Commit graph speeds up traversal dramatically:
$ git commit-graph write --reachable
# Stores precomputed generation numbers and tree SHAs
```

■ *For very large repos, git maintenance is superior to git gc. It runs incrementally in background without blocking other operations.*

1.13 Git Hooks

Git hooks are scripts executed at specific points in the Git lifecycle. They live in `.git/hooks/` and are NOT version-controlled by default (though tools like Husky, Lefthook, and pre-commit manage this).

Hook	When	Typical Use
pre-commit	Before commit is created	Lint, format, unit tests
prepare-commit-msg	Before editor opens	Inject ticket number from branch
commit-msg	After message entered	Enforce conventional commits
post-commit	After commit created	Notifications, ticket updates
pre-push	Before git push	Run full test suite
pre-receive	Server: before refs update	Policy enforcement
update	Server: per-ref update	Per-branch policy
post-receive	Server: after refs update	CI trigger, notifications
pre-rebase	Before rebase starts	Prevent rebase of shared branches

```
# Example: pre-commit hook enforcing no debug statements
#!/bin/bash
if git diff --cached --name-only | xargs grep -l 'console\.log\|pdb\.set_trace' 2>/dev/null; then
    echo "ERROR: Debug statements found. Remove before committing."
    exit 1
fi

# Example: commit-msg enforcing Conventional Commits
#!/bin/bash
commit_msg=$(cat "$1")
pattern='^(feat|fix|docs|style|refactor|test|chore)(\.(.+))?: .+'
if ! echo "$commit_msg" | grep -qE "$pattern"; then
    echo "ERROR: Commit message must follow Conventional Commits spec"
    echo "  Example: feat(auth): add OIDC login flow"
    exit 1
fi
```

■ *Server-side hooks (pre-receive, update, post-receive) enforce policy at the Git server level and cannot be bypassed by client-side hook manipulation. GitHub implements its own server-side hooks for branch protection rules.*

1.14 Sparse Checkout & Partial Clone

Sparse Checkout

Sparse checkout allows working with only a subset of a repository's files, reducing disk usage and checkout time. Essential for monorepos.

```
# Enable sparse checkout:
$ git clone --no-checkout https://github.com/org/monorepo
$ cd monorepo
$ git sparse-checkout init --cone

# Specify patterns (cone mode - directory based):
$ git sparse-checkout set services/auth services/api libs/shared

# Now checkout only populates those directories:
$ git checkout main

# Disable (get all files):
$ git sparse-checkout disable

# Advanced: non-cone mode (glob patterns):
$ git sparse-checkout set --no-cone '!/generated' '/src' '/tests'
```

Partial Clone

Partial clone defers downloading of objects until they're needed, dramatically reducing initial clone time for large repositories.

```
# Blobless clone (fetches all commits and trees, but not blobs until needed):
$ git clone --filter=blob:none https://github.com/org/large-repo

# Treeless clone (fetches commits, but not trees or blobs until needed):
$ git clone --filter=tree:0 https://github.com/org/large-repo

# The most aggressive (fastest initial clone, most deferred):
$ git clone --filter=tree:0 --no-checkout https://github.com/org/large-repo

# Combine with sparse checkout for maximum efficiency in monorepos:
$ git clone --filter=blob:none --sparse https://github.com/org/monorepo
$ git sparse-checkout set services/auth
```

■ *GitHub's infrastructure supports partial clone via the protocol v2 with filter capabilities. This is how GitHub.com itself handles git clone for very large repos like the VS Code repository.*

1.15 Git LFS (Large File Storage)

Git LFS replaces large binary files in the repository with small text pointer files, while storing the actual content on a separate LFS server. This keeps the main repository small while supporting large assets.

```
# Install and initialize LFS:
$ git lfs install
$ git lfs track '*.psd' '*.zip' '*.tar.gz' '*.mp4'

# The .gitattributes file is created:
$ cat .gitattributes
*.psd filter=lfs diff=lfs merge=lfs -text
*.zip filter=lfs diff=lfs merge=lfs -text

# Pointer file stored in git (what the repo contains):
version https://git-lfs.github.com/spec/v1
oid sha256:9f86d08...
size 12345678

# Actual binary stored on LFS server (GitHub's LFS endpoint):
# https://github.com/org/repo.git/info/lfs

# Check LFS status:
$ git lfs ls-files
$ git lfs status
```

■ *GitHub LFS storage and bandwidth are billed separately. Large orgs with many binaries should consider dedicated artifact stores (Artifactory, AWS S3 + CloudFront) and use LFS only for files that must be version-controlled alongside code.*

1.16 Submodules vs Subtrees

Submodules

Submodules pin a specific commit of an external repository inside your repository. The parent repo stores a gitlink (160000 mode entry) pointing to the SHA of the submodule commit.

```
# Add a submodule:
$ git submodule add https://github.com/org/shared-lib libs/shared
$ cat .gitmodules
[submodule "libs/shared"]
  path = libs/shared
  url = https://github.com/org/shared-lib

# Clone with submodules:
$ git clone --recurse-submodules https://github.com/org/app

# Update submodules:
$ git submodule update --remote --merge

# Common gotcha: submodule is a detached HEAD in the parent!
$ cd libs/shared && git status
HEAD detached at a3b2c1d
```

Subtrees

Subtree merges embed another repository's history into a subdirectory of your repository. Unlike submodules, there's no separate .gitmodules file and contributors don't need to run submodule init.

```
# Add a subtree:
$ git subtree add --prefix=libs/shared https://github.com/org/shared-lib main --squash

# Pull updates:
$ git subtree pull --prefix=libs/shared https://github.com/org/shared-lib main --squash

# Push changes back:
$ git subtree push --prefix=libs/shared https://github.com/org/shared-lib main
```

Criterion	Submodules	Subtrees
Complexity	High (requires submodule commands)	Lower (standard git)
Contributor friction	High (must init submodules)	None
Repository history	External ref only	Embedded in parent history
Storage overhead	Minimal (pointer)	Full history duplicated
Independent updates	Easy (submodule update)	Manual subtree push/pull
CI complexity	Needs --recurse-submodules	Transparent
Enterprise verdict	Avoid unless necessary	Prefer for internal libs

1.17 Monorepo Support & Large Repository Optimization

Monorepo Architecture

A monorepo stores multiple projects in a single repository. Google, Meta, Microsoft, and Airbnb all run multi-million file monorepos. Git was not designed for this scale, requiring significant tooling investment.

Techniques for Large Repos

- **Partial Clone + Sparse Checkout:** The gold standard. Developers clone only the trees and blobs for their service.
- **Commit Graph:** Precomputes generation numbers so log, blame, and merge-base operations skip $O(n)$ traversals.
- **FSMONITOR:** Filesystem monitor daemon (Watchman integration) for fast status/add by watching inode events instead of `stat()` on every file.
- **Multi-Pack Index (MIDX):** Single index covering all pack files, improving object lookup performance.
- **Geometric Repacking:** Maintains pack files in geometric size series, minimizing repacking work.

```
# Enable all performance features:
$ git config feature.manyFiles true # enables fsmonitor + index.skipHash
$ git config core.fsmonitor true
$ git maintenance start

# Enable commit graph writes automatically:
$ git config fetch.writeCommitGraph true

# Multi-pack index:
$ git multi-pack-index write

# Geometric repacking:
$ git repack --geometric=2 -d
```

VFS for Git (GVFS / Scalar)

Microsoft developed VFS for Git (later Scalar) for the Windows OS repository — 300GB, 3.5M files. Scalar is now upstream in Git 2.38+.

```
# Scalar provides an opinionated setup of all performance features:
$ scalar clone https://github.com/org/massive-monorepo
# Automatically enables:
# - Partial clone (blob:none)
# - Sparse checkout
# - Background maintenance
# - Filesystem monitor
# - Commit graph
# - Multi-pack index
```

Anti-Patterns for Large Repos

- Storing large binaries in Git (use LFS or an artifact store)
- Committing generated files that change on every build
- Running git status on a 3M-file repo without fsmonitor
- Full clone in CI without caching or partial clone
- Not enabling commit-graph, leading to $O(n)$ merge-base traversals

- Using `git gc --aggressive` in automated pipelines (very slow)

Interview Questions — Git Internals

Q: What is the difference between a Git blob and a tree?

A: A blob stores raw file content (no filename). A tree is a directory: it contains references (SHA+mode+name) to blobs and other trees. Filenames live in trees, not blobs.

Q: Why does git rebase change commit SHAs?

A: A commit SHA is determined by its content: tree SHA, parent SHA, author, committer, timestamp, and message. Rebasing creates new commits with a different parent SHA, so all downstream SHAs change.

Q: Explain the staging area (index). Why does Git have it?

A: The index is a binary file representing the 'proposed next commit.' It decouples 'what files to include' from 'when to commit,' enabling partial staging, format-patch workflows, and efficient tree reconstruction without touching the working directory.

Q: When would you use git revert over git reset?

A: Always use revert on shared/protected branches because it creates a new commit that undoes changes without altering existing history. Use reset only on local or private branches where rewriting history is safe.

Q: How does Git LFS work internally?

A: LFS installs a Git smudge/clean filter. On checkout (smudge), it replaces the pointer file with the actual blob fetched from the LFS server. On staging (clean), it uploads the blob and writes a pointer. The main repo never contains the binary.

Q: What is a partial clone and when should you use it?

A: Partial clone defers downloading of blobs (`--filter=blob:none`) or trees (`--filter=tree:0`) until they're accessed. Use it in CI pipelines and large monorepos to reduce clone times from minutes to seconds.